How LLMs Are Built

Pre-Work Module 09 (Optional)

ECBS5200 — Practical Deep Learning Engineering

Why this matters for you

This course is about **post-training engineering.** You receive a pretrained model and adapt it.

But that model didn't appear from nothing. Understanding how it was built helps you:

- Know what the model already knows (and what it doesn't)
- Understand why a 494M decoder can beat a 149M encoder
- Make better decisions about which model to start from
- Speak the language of the field you're entering

The pipeline: four stages

```
1. Pre-training          Learns language from raw text
        ↓
2. Mid-training          (Optional) Domain or capability adaptation
        ↓
3. Post-training         Instruction tuning, alignment
        ↓
4. Your fine-tuning      What this course teaches
```

Most of the intelligence comes from stage 1. Stages 2-3 shape the behavior. Stage 4 is you.

Stage 1: Pre-training

The goal: learn to predict text. The model learns language as a side effect.

Model	Year	Parameters	Training tokens	Estimated cost
GPT-3	2020	175B	300B	~$4.6M
Llama 2	2023	70B	2T	~$2M+
Llama 3	2024	405B	15T	~$60-100M
Qwen 2.5	2024	0.5B–72B	18T	undisclosed
ModernBERT	2024	149M	2T	undisclosed

In four years: training data went from 300B to 18T tokens (60x). Costs went from $5M to $100M+.

Encoders vs decoders: why both exist

	Encoder	Decoder
Reads text	All at once (bidirectional)	Left to right (causal mask)
Pre-training task	Fill in masked words	Predict next word
Good at	Understanding, classification	Generation, and also classification
For classification	One forward pass	Also one forward pass (with classification head)
Example	ModernBERT (149M)	Qwen 2.5 (0.5B–72B)

For classification, the speed difference comes from **model size**, not architecture. A 494M decoder is ~3x slower than a 149M encoder because it has 3x more parameters — not because it's a decoder.

Scaling laws: why bigger models are better

In 2020, OpenAI published a finding that changed the field:

Model quality scales predictably with three things:

1. Number of parameters
2. Amount of training data
3. Amount of compute

Double the compute → predictable improvement in quality. No plateaus, no diminishing returns (within the range studied).

The Chinchilla insight (2022): most models were too big for their data. A smaller model trained on more data beats a larger model trained on less.

What scaling laws mean for this course

You will work with two models:

- **ModernBERT-base:** 149M parameters, trained on 2T tokens
- **Qwen2.5-0.5B:** 494M parameters, trained on 18T tokens

The decoder has **3.3x more parameters** trained on **9x more data.**

Scaling laws predict it will have richer representations. That's exactly what we observe: the decoder, with LoRA on 0.46% of its parameters, beats the fully fine-tuned encoder on rare-class performance.

The encoder's advantage? It's smaller and therefore faster. Scaling laws say nothing about inference speed.

Stage 2: Mid-training

Same training process, different data diet.

The model keeps predicting tokens, but the training data shifts to emphasize specific capabilities:

Llama 3 → 3.1: Meta continued training on progressively longer documents (16K → 32K → 128K tokens). Same loss function, longer inputs. Turned an 8K-context model into a 128K-context model without starting over.

Llama → CodeLlama: Continued pre-training on 500B+ tokens of code. The model already knew English; now it gets much better at Python and Java.

Qwen 2.5: Alibaba continued pre-training on math, code, and multilingual data. That's why Qwen 2.5 is strong on math benchmarks despite being a general-purpose model.

Stage 3: Post-training alignment

After pre-training, the model can predict text but doesn't know how to be helpful.

Instruction tuning: train on (instruction, response) pairs so the model follows directions.

RLHF / DPO: use human preferences to align the model's behavior — helpful, harmless, honest.

This is why `Qwen2.5-0.5B-Instruct` exists alongside `Qwen2.5-0.5B`:

- **Base:** raw pre-trained model. Good representations but doesn't follow instructions.
- **Instruct:** post-trained to be a helpful assistant. Better at chat, worse for classification heads.

For classification with LoRA, we use Base — we want the representations, not the chat behavior.

Stage 4: Your fine-tuning

This is where you come in.

You receive a model that already understands language. You teach it your specific task:

- **Full fine-tuning:** update all parameters (Weeks 1-2)
- **LoRA:** update <1% of parameters (Week 3)
- **Quantization:** compress without retraining (Week 5)
- **Distillation:** transfer knowledge between models (Week 6)

The model arrives knowing English, world knowledge, and reasoning. You just need to teach it 113 complaint categories.

Key takeaways

1. **Pre-training** is where models learn language — from trillions of tokens, at enormous cost.
2. **Scaling laws** predict that more parameters + more data = better quality. The 494M decoder was trained on 9x more data than the 149M encoder.
3. **Encoders** read bidirectionally (fast, good for classification). **Decoders** read left-to-right (slower, richer representations).
4. **Post-training** (instruction tuning, RLHF) shapes behavior. For classification, we use Base models, not Instruct.
5. **Your fine-tuning** is the last mile. The model already knows language. You teach it your task.

Further reading (optional)

Paper	What it's about	Year
Vaswani et al., "Attention Is All You Need"	The Transformer architecture	2017
Devlin et al., "BERT"	Encoder pre-training for NLP	2018
Radford et al., "Language Models are Unsupervised Multitask Learners" (GPT-2)	Decoder pre-training at scale	2019
Kaplan et al., "Scaling Laws for Neural Language Models"	Why bigger = better	2020
Hoffmann et al., "Chinchilla"	Optimal compute allocation	2022
Ouyang et al., "InstructGPT"	RLHF alignment	2022
Shoeybi et al., "Megatron-LM"	Efficient large-scale training	2020

None of these are required. They're here if you want to go deeper.